

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE TEORIA

INFORMACIÓN BÁSICA					
ASIGNATURA:	FISICA COMPUTACIONAL				
TÍTULO DE LA PRÁCTICA:	Resolución de la Práctica 1: Variante A				
NÚMERO DE LA PRÁCTICA:	01	AÑO LECTIVO:	2026	NRO. SEMESTRE:	VII
FECHA DE PRESENTACIÓN:	03/05/2026	HORA DE PRESENTACIÓN:	11:59:00		
INTEGRANTE(s): - Jara Mamani Mariel Alisson				NOTA (0 - 20):	Nota colocada por el docente
DOCENTE: Prof. Edwin Agapito Llamoca Requena					

RESULTADOS Y PRUEBAS
SOLUCIÓN DE LOS EJERCICIOS PROPUESTOS VARIANTE A 1 $\vec{a} = 0$, no hay aceleración 1.1 Velocidad Constante en una Dimensión 1.1.1 Problema Sea una partícula, cuya posición inicial es $t_0 = 0$, $x_0 = 5$ m con $v_x = -2$ m/s. Determine la posición de la partícula para $t = 10$ s. Utilice el tamaño de paso $h = 0.01$. Hacia donde se dirige la partícula. Implementación: Se definieron las variables iniciales $x=5$, $v=-2$, $a=0$, $h=0.01$. Se utilizó un ciclo for para calcular la posición en cada instante t con el paso h establecido y se graficó según lo que se pide. Código fuente: exe111.py <pre>import matplotlib.pyplot as plt import numpy as np # Constantes del sistema k=1 m=2 h=0.01 a=0 # Condiciones iniciales t=0 x=5 v=-2 tfin=10 # Listas para almacenar los resultados pt = [t] pv = [v] px = [x] pa = [a] for t in np.arange(0, tfin, h): # Formulas v = v + h*a</pre>

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

```

x = x + h*v

pt.append(t)
px.append(x)
pv.append(v)
pa.append(a)

# Graficas
plt.figure(figsize=(12, 8))

# Grafica 1: Aceleración vs Tiempo
plt.subplot(2, 2, 1)
plt.plot(pt, pa, label='Aceleración', color='red')
plt.title('Aceleración vs Tiempo')
plt.xlabel('Tiempo (s)')
plt.ylabel('Aceleración (m/s²)')

plt.grid(True)

# Grafica 2: Velocidad vs Tiempo
plt.subplot(2, 2, 2)
plt.plot(pt, pv, label='Velocidad', color='blue')
plt.title('Velocidad vs Tiempo')
plt.xlabel('Tiempo (s)')
plt.ylabel('Velocidad (m/s)')
plt.grid(True)

# Grafica 3: Posición vs Tiempo
plt.subplot(2, 2, 3)
plt.plot(pt, px, label='Posición', color='green')
plt.title('Posición vs Tiempo')
plt.xlabel('Tiempo (s)')
plt.ylabel('Posición (m)')
plt.grid(True)

# Grafica 4: Velocidad vs Posición
plt.subplot(2, 2, 4)
plt.plot(px, pv, label='Fase', color='purple')
plt.title('Posición vs Velocidad')
plt.xlabel('Posición (m)')
plt.ylabel('Velocidad (m/s)')
plt.grid(True)

# Ajustar los márgenes para evitar solapamientos
plt.tight_layout()
plt.savefig('TAREA-01/img/pr_a/exe111.png')
plt.show()

```

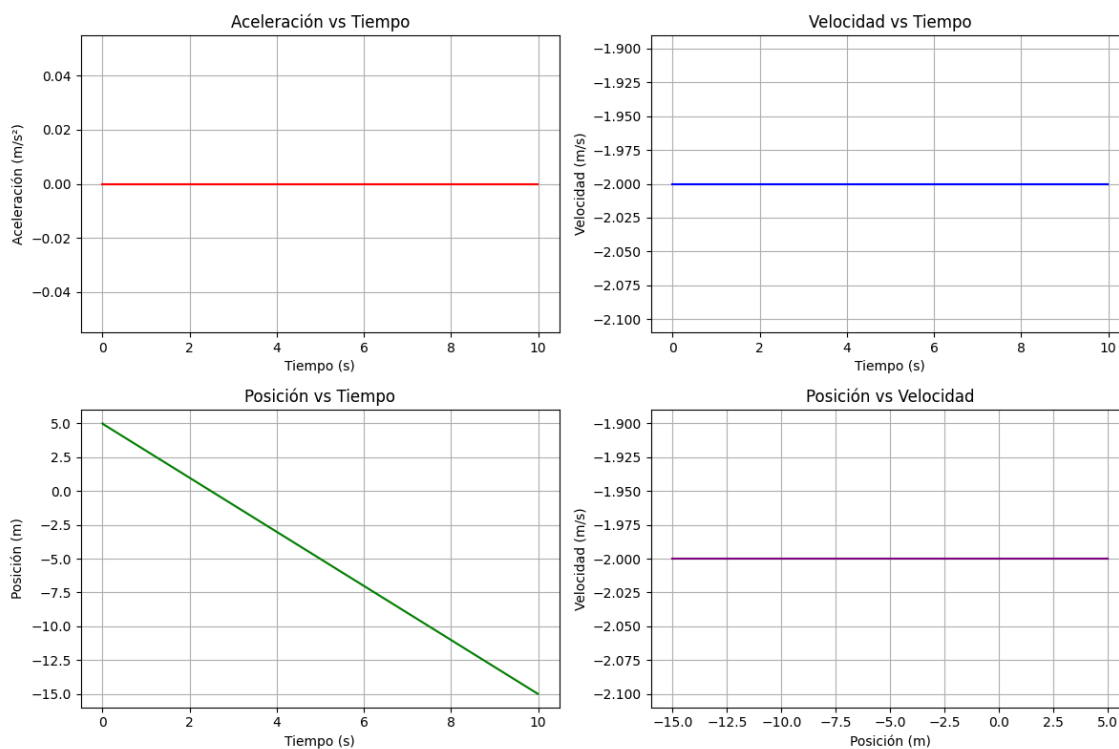


Figura 1: Diagramas $a-t$, $v-t$, $x-t$, $v-x$.

Resultados: En el gráfico $a-t$ se observa una línea horizontal en $a=0$ porque la aceleración es nula. En $v-t$ se observa una línea recta con pendiente cero debido a que la velocidad es constante. En $x-t$ se observa una línea recta con pendiente negativa porque la partícula se mueve hacia la izquierda. En $v-x$ se observa un punto único porque la velocidad no cambia. Concluyendo que la partícula se dirige hacia la izquierda.

1.1.2 Problema

La posición inicial de una partícula se ubica en $x_0 = 5$ m y $t_0 = 0$ con velocidad $v_x = -3$ m/s. Después de 3 s, adquiere una velocidad $v_x = 2$ m/s transcurriendo 2 s más. Encuentre la posición final de la partícula. Utilice tamaño de paso $h = 0.1$.

Implementación: Se definieron las condiciones iniciales $x=5$, $v=-3$, $h=0.1$. Se utilizó un ciclo for con una condición para cambiar la velocidad cuando $t > 3$ s.

Código fuente: exe112.py

```
import matplotlib.pyplot as plt
import numpy as np

# Constantes del sistema
k=1
m=2
h=0.1
a=0

# Condiciones iniciales
t=0
x=5
v=-3
tfin=3+2

# Listas para almacenar los resultados
pt = [t]
pv = [v]
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

```

px = [x]
pa = [a]

for t in np.arange(0, tfin, h):

    if t > 3:
        v = 2

    # Formulas
    v = v + h*a
    x = x + h*v

    pt.append(t)
    px.append(x)
    pv.append(v)
    pa.append(a)

# Graficas
plt.figure(figsize=(12, 8))

# Grafica 1: Aceleración vs Tiempo
plt.subplot(2, 2, 1)
plt.plot(pt, pa, label='Aceleración', color='red')
plt.title('Aceleración vs Tiempo')
plt.xlabel('Tiempo (s)')
plt.ylabel('Aceleración (m/s²)')
plt.grid(True)

# Grafica 2: Velocidad vs Tiempo
plt.subplot(2, 2, 2)
plt.plot(pt, pv, label='Velocidad', color='blue')
plt.title('Velocidad vs Tiempo')
plt.xlabel('Tiempo (s)')
plt.ylabel('Velocidad (m/s)')
plt.grid(True)

# Grafica 3: Posición vs Tiempo
plt.subplot(2, 2, 3)
plt.plot(pt, px, label='Posición', color='green')
plt.title('Posición vs Tiempo')
plt.xlabel('Tiempo (s)')
plt.ylabel('Posición (m)')
plt.grid(True)

# Grafica 4: Velocidad vs Posición
plt.subplot(2, 2, 4)
plt.plot(px, pv, label='Fase', color='purple')
plt.title('Posición vs Velocidad')
plt.xlabel('Posición (m)')
plt.ylabel('Velocidad (m/s)')
plt.grid(True)

# Ajustar los márgenes para evitar solapamientos
plt.tight_layout()
plt.savefig('TAREA-01/img/pr_a/exe112.png')
plt.show()

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

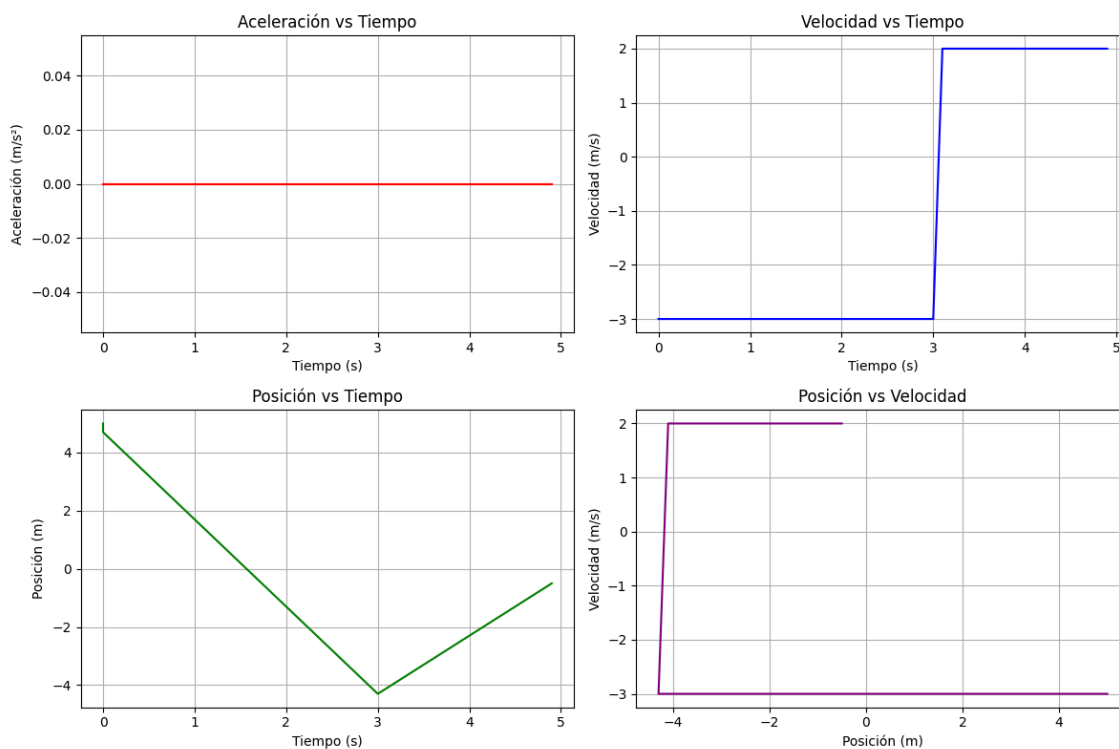


Figura 2: Diagramas $a-t$, $v-t$, $x-t$, $v-x$.

Resultados: En el gráfico $a-t$ se observa una línea horizontal porque la aceleración es cero. En $v-t$ se observa un escalón debido al cambio de velocidad aumenta en $t > 3s$. En $x-t$ se observa un cambio de pendiente cuando la velocidad cambia de dirección. En $v-x$ se observa una línea horizontal en cada intervalo.

1.2 Velocidad constante en dos dimensiones

1.2.1 Problema

Una partícula parte del reposo desde $\vec{r} = (3\hat{i} + 4\hat{j})$ m con $\vec{v} = -(2\hat{i} + 4\hat{j})$ m/s. Encuentre su posición final y el desplazamiento cuando $t = 5$ s.

Implementación: Se definieron los vectores posición inicial $\vec{r}_i$ y velocidad $\vec{v}$ con paso $h=0.1$. Se utilizó un ciclo para actualizar la posición usando la fórmula vectorial. Se representó los vectores $\vec{r}_i$, $\vec{r}_f$ y $\Delta \vec{r}$ usando quiver.

Código fuente: exe121.py

```
import matplotlib.pyplot as plt
import numpy as np

# Constantes del sistema
h=0.1
a=0
# Condiciones iniciales
t=0
tfin=5

ri = np.array([3, 4])
v = np.array([-2, -4])
r_actual = ri.copy()
trayectoria = [r_actual.copy()]

for t in np.arange(0, tfin, h):
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

```
# Formulas
r_actual = r_actual + (v * h)
trayectoria.append(r_actual.copy())

trayectoria = np.array(trayectoria)
rf = trayectoria[-1]
dr = rf - ri

# Visualización
plt.figure(figsize=(8, 8))

# 1. Dibujar los puntos de la trayectoria
plt.plot(trayectoria[:, 0], trayectoria[:, 1], 'k.', markersize=5, label='Puntos Trayectoria')

# 2. Vector Posición Inicial (ri) - Desde el origen (0,0)
plt.quiver(0, 0, ri[0], ri[1], angles='xy', scale_units='xy', scale=1, color='blue', label='$r_i$', width=0.01)

# 3. Vector Desplazamiento (Delta r) - Desde ri hasta rf
plt.quiver(ri[0], ri[1], dr[0], dr[1], angles='xy', scale_units='xy', scale=1, color='red', label='$\Delta r$', width=0.01)

# 4. Vector Posición Final (rf) - Desde el origen (0,0)
plt.quiver(0, 0, rf[0], rf[1], angles='xy', scale_units='xy', scale=1, color='green', label=r'$r_f$', alpha=0.5, width=0.008)

# Graficación
plt.grid(True)
plt.axis('equal')
plt.title('Ejercicio 1.2.1 - Trayectoria y Vectores')
plt.xlabel('Eje X (m)')
plt.ylabel('Eje Y (m)')
plt.legend()

print(f"Posición Final (rf): {rf}")
print(f"Desplazamiento (Δr): {dr}")

plt.tight_layout()
plt.savefig('TAREA-01/img/pr_a/exe121.png')
plt.show()
```

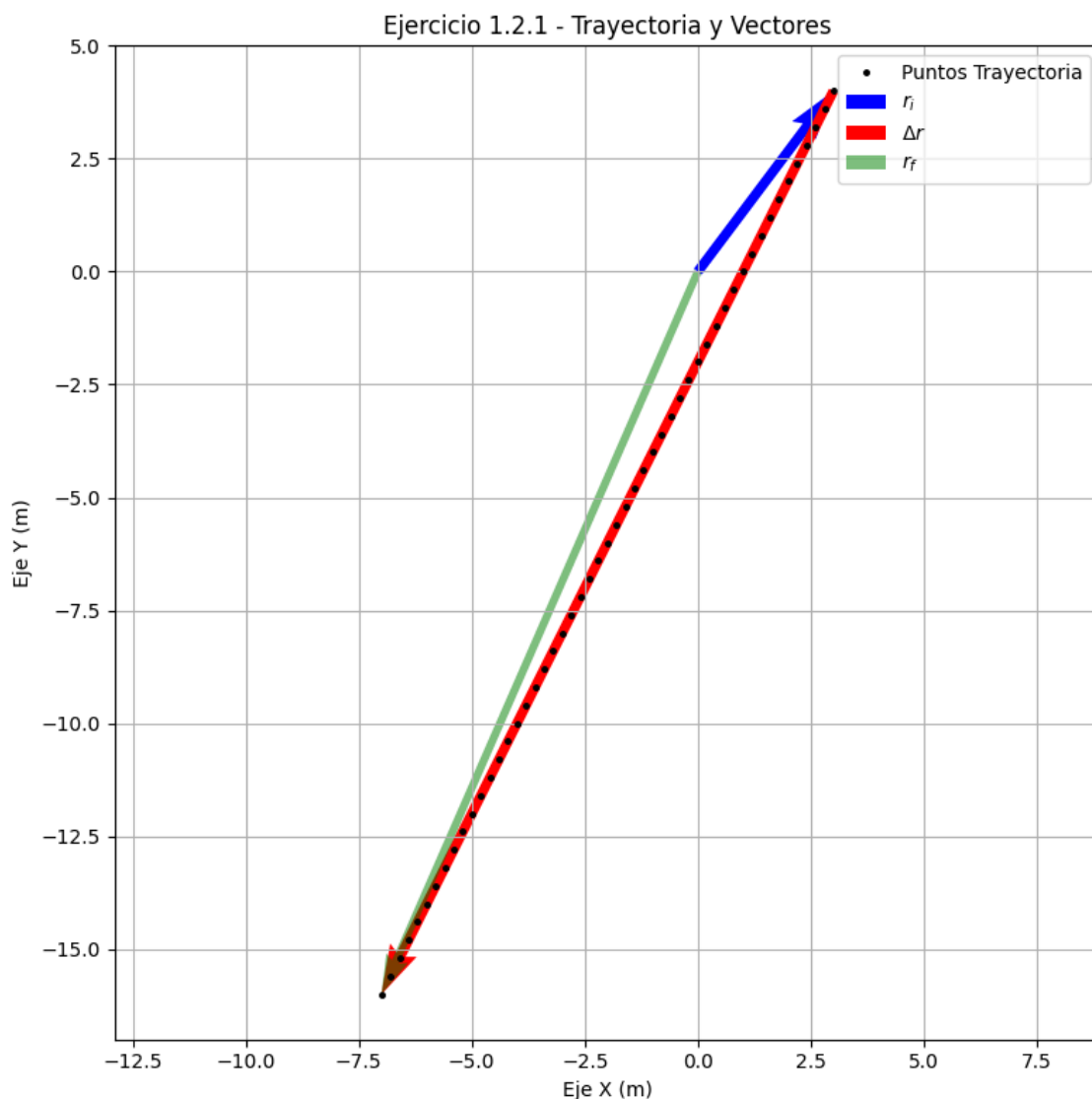


Figura 3: Trayectoria en dos dimensiones.

Resultados: En el gráfico se observa la trayectoria de la partícula con los vectores de posición inicial (azul), desplazamiento (rojo) y posición final (verde).

1.2.2 Problema

Una partícula parte del reposo desde $\vec{r} = (3\hat{i} + 4\hat{j})$ m con una velocidad $\vec{v} = -(4\hat{i} + 3\hat{j})$ m/s. Cuando inicia su movimiento se presenta una velocidad del viento $\vec{v}_v = (3\hat{i} - 5\hat{j})$ m/s. Encuentre su posición final y el desplazamiento cuando $t = 10$ s.

Implementación: Se calculó la velocidad resultante $v_t = v_p + v_w$. Se utilizó un ciclo para actualizar la posición con la fórmula vectorial y se representó los vectores usando quiver.

Código fuente: exe122.py

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

```
import matplotlib.pyplot as plt
import numpy as np

# Constantes del sistema
h=0.1
a=0
# Condiciones iniciales
t=0
tfin=10

ri = np.array([3, -4])
vp = np.array([-4, 3])
vw = np.array([3,-5])
vt = vp + vw

r_actual = ri.copy()
trayectoria = [r_actual.copy()]

for t in np.arange(0, tfin, h):

    # Formulas
    r_actual = r_actual + (vt * h)
    trayectoria.append(r_actual.copy())

trayectoria = np.array(trayectoria)
rf = trayectoria[-1]
dr = rf - ri

# Visualización
plt.figure(figsize=(8, 8))

# 1. Dibujar los puntos de la trayectoria
plt.plot(trayectoria[:, 0], trayectoria[:, 1], 'k.', markersize=5, label='Puntos Trayectoria')

# 2. Vector Posición Inicial (ri) - Desde el origen (0,0)
plt.quiver(0, 0, ri[0], ri[1], angles='xy', scale_units='xy', scale=1, color='blue', label='$r_i$', width=0.01)

# 3. Vector Desplazamiento (Delta r) - Desde ri hasta rf
plt.quiver(ri[0], ri[1], dr[0], dr[1], angles='xy', scale_units='xy', scale=1, color='red', label='$\Delta r$', width=0.01)

# 4. Vector Posición Final (rf) - Desde el origen (0,0)
plt.quiver(0, 0, rf[0], rf[1], angles='xy', scale_units='xy', scale=1, color='green', label=r'$r_f$', alpha=0.5, width=0.008)

# Graficación
plt.grid(True)
plt.axis('equal')
plt.title('Ejercicio 1.2.1 - Trayectoria y Vectores')
plt.xlabel('Eje X (m)')
plt.ylabel('Eje Y (m)')
plt.legend()

print(f"Posición Final (rf): {rf}")
print(f"Desplazamiento (Δr): {dr}")

plt.tight_layout()
plt.savefig('TAREA-01/img/pr_a/exe122.png')
plt.show()
```


Ejercicio 1.2.1 - Trayectoria y Vectores

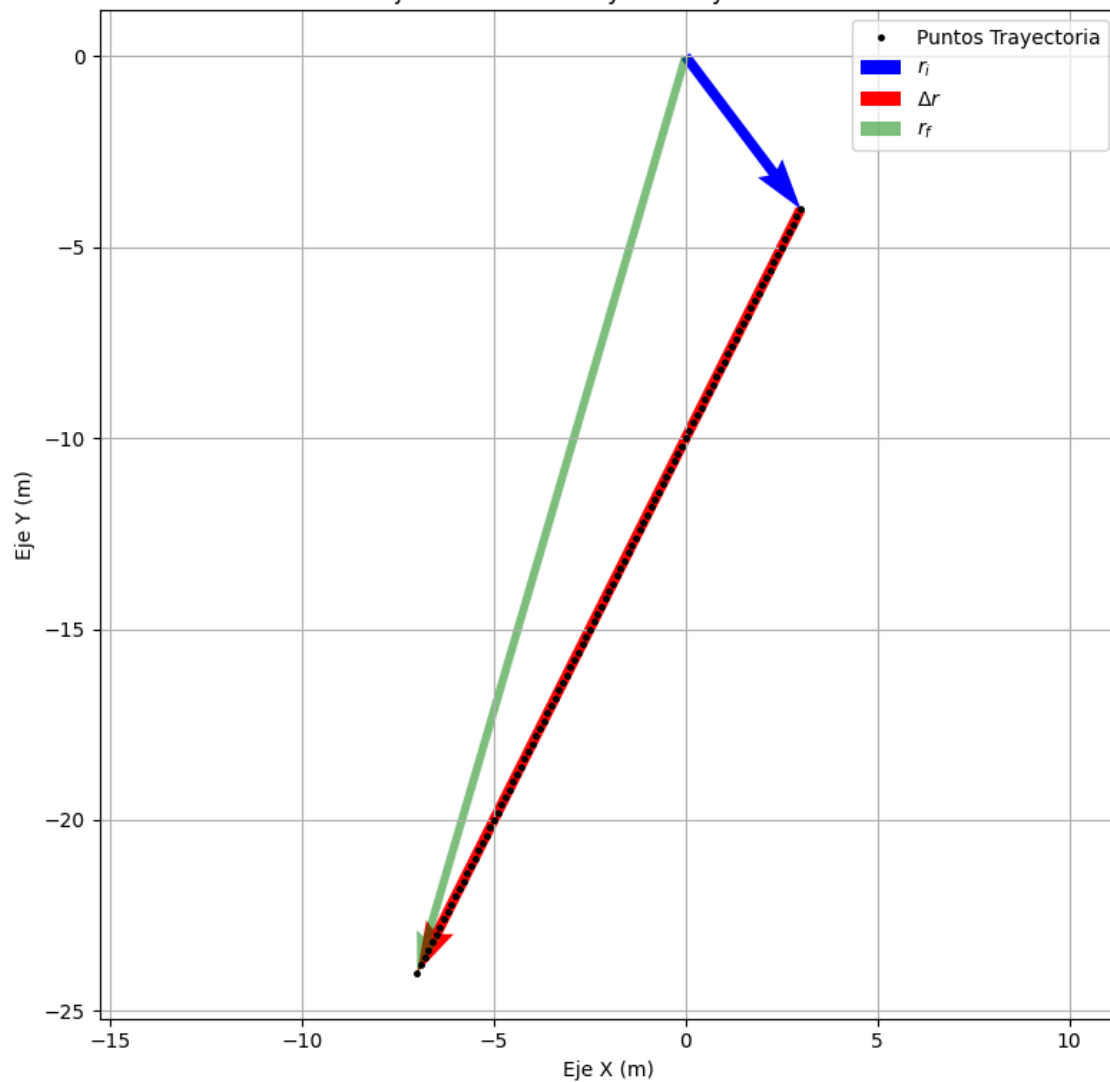


Figura 4: Trayectoria en dos dimensiones con velocidad del viento.

Resultado: Se observa la trayectoria con vectores r_i , r_f y Δr considerando la velocidad del viento.

1.3 Velocidad constante en tres dimensiones

1.3.1 Problema

Una partícula parte del reposo desde $\vec{r} = (3\hat{i} - 4\hat{j} - 5\hat{k})$ m con $\vec{v} = (-2\hat{i} + 4\hat{j} + 6\hat{k})$ m/s. Cuando inicia su movimiento se presenta una velocidad del viento $\vec{v}_v = (-3\hat{j} + 5\hat{k})$ m/s. Encuentre su posición final y el desplazamiento cuando $t = 10$ s.

Implementación: Se calculó la velocidad resultante $\vec{v}_t = \vec{v}_p + \vec{v}_w$ en 3D. Se representó los vectores r_i , r_f y Δr usando quiver en proyección 3D.

Código fuente: exe131.py

```
import matplotlib.pyplot as plt
import numpy as np
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10

```
# 1. Parámetros de simulación
h=0.1
a=0
t=0
tfin=10

ri = np.array([3, -4, -5])
vp = np.array([-2, 4, 6])
vw = np.array([0, -3, 5])
vt = vp + vw

r_actual = ri.copy()
trayectoria = [r_actual.copy()]

for t in np.arange(0, tfin, h):

    # 2. Formulas
    r_actual = r_actual + (vt * h)
    trayectoria.append(r_actual.copy())

trayectoria = np.array(trayectoria)
rf = trayectoria[-1]
dr = rf - ri

# 3. Visualización
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

# Definir etiquetas con coordenadas para cada vector.// Formatear decimales a 2 dígitos para mejor legibilidad.
label_ri = f"ri ({ri[0]:.0f}, {ri[1]:.0f}, {ri[2]:.0f})"
label_dr = f"Δr ({dr[0]:.0f}, {dr[1]:.0f}, {dr[2]:.0f})"
label_rf = f"rf ({rf[0]:.0f}, {rf[1]:.0f}, {rf[2]:.0f})"

# 1. Puntos de la trayectoria
# Poner puntos invisibles en los extremos para "estirar" los ejes para evitar solapamiento
ax.scatter([0, ri[0], rf[0]], [0, ri[1], rf[1]], [0, ri[2], rf[2]], alpha=0.0)

# 2. Vector Posición Inicial (ri) - Desde el origen (0,0,0)
ax.quiver(0, 0, 0, ri[0], ri[1], ri[2], color='blue', pivot='tail', label=r'$\vec{r}_i$', arrow_length_ratio=0.5)
ax.text(ri[0]+0.5, ri[1], ri[2], label_ri, color='blue', fontsize=10)

# 3. Vector Desplazamiento (Delta r) - Desde ri hasta rf
ax.quiver(ri[0], ri[1], ri[2], dr[0], dr[1], dr[2], color='red', linestyle='--', label=r'$\Delta \vec{r}$', arrow_length_ratio=0.08)
ax.text(ri[0]+dr[0]/2, ri[1]+dr[1]/2, ri[2]+dr[2]/2, label_dr, color='red', fontsize=10)

# 4. Vector Posición Final (rf) - Desde el origen (0,0,0)
ax.quiver(0, 0, 0, rf[0], rf[1], rf[2], color='green', label=r'$\vec{r}_f$', arrow_length_ratio=0.08)
ax.text(rf[0], rf[1], rf[2], label_rf, color='green', fontsize=10)

# --- CONFIGURACIÓN DE PANTALLA ---
ax.set_xlabel('Eje X (m)')
ax.set_ylabel('Eje Y (m)')
ax.set_zlabel('Eje Z (m)')
ax.set_title('Ejercicio 1.3: Movimiento en 3D con Viento')
ax.legend()

# Resultados por terminal
print(f"Velocidad Resultante: {vt} m/s")
print(f"Posición Final (rf): {rf} m")
print(f"Desplazamiento (dr): {dr} m")

plt.tight_layout()
plt.savefig('TAREA-01/img/pr_a/exe131.png')
plt.show()
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 11

Ejercicio 1.3: Movimiento en 3D con Viento

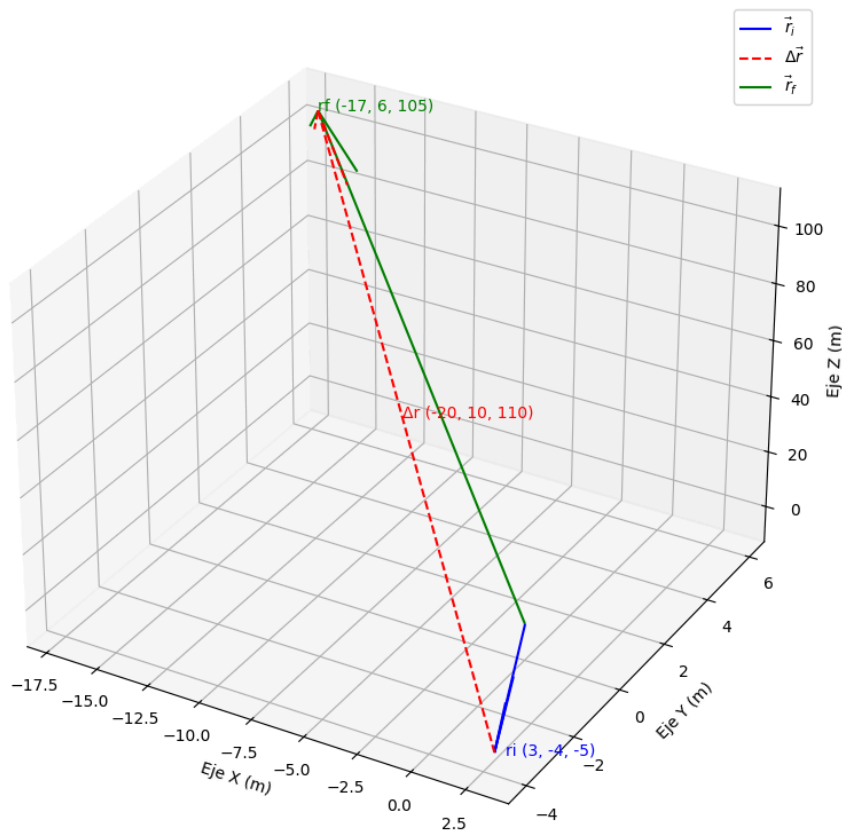


Figura 5: Trayectoria en tres dimensiones.

Descripción: Se observa la trayectoria en 3D con los vectores de posición inicial, desplazamiento y posición final.

2 Caída Libre $a_y = -10\hat{j}$,

2.1 En una dimensión

2.1.1 Problema

Una pelota se lanza con $v_y = 10$ m/s desde $y = 2$ m. Determine la altura máxima, cuando la velocidad sea cero, el tiempo donde llega a la altura máxima, el tiempo donde pasa por $y = 2$ y el tiempo en que llega al suelo.

Implementación: Se definieron las condiciones iniciales $y=2$, $v_y=10$, $a_y=-10$, $h=0.01$. Se utilizó un ciclo `while` que continúa hasta que $y \geq 0$. Se calculó la altura máxima usando `np.argmax`.

Código fuente: exe211.py

```
import matplotlib.pyplot as plt
import numpy as np

# 1. Constantes del sistema
h = 0.01
ay=-10

# Condiciones iniciales
t=0
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 12

```

y=2
vy=10

# Listas para guardar datos
pt = [t]
py = [y]
pvy = [vy]
pa = [ay]

# 2. TRAYECTORIA (Simulación hasta que llega al suelo y < 0)
while y >= 0:

    # Fórmulas
    vy = vy + (h * ay)
    y = y + (h * vy)
    t = t + h

    # Guardar datos
    pt.append(t)
    py.append(y)
    pvy.append(vy)
    pa.append(ay)

pt, py, pvy, pa = np.array(pt), np.array(py), np.array(pvy), np.array(pa)

# 3. ANÁLISIS DE RESULTADOS
idx_max = np.argmax(py)
h_max = py[idx_max]
t_h_max = pt[idx_max]
t_suelo = pt[-1]

# Punto donde regresa ay y=2
idx_y2 = np.argmin(np.abs(py[pt > 0.5] - 2)) + np.where(pt > 0.5)[0][0]
t_y2= pt[idx_y2]

# 4. VISUALIZACIÓN
fig, axs = plt.subplots(2, 2, figsize=(12, 10))
plt.gcf().canvas.manager.set_window_title('Laboratorio 2: Caída Libre')

# 1. ACELERACIÓN vs TIEMPO
axs[0,0].plot(pt, pa, 'r', linewidth=2)
axs[0,0].set_title('Aceleración vs Tiempo')
axs[0,0].set_ylabel('ay (m/s²)')
axs[0,0].set_xlabel('t (s)')
axs[0,0].grid(True)
# Punto ay=-10
axs[0,0].scatter(pt[len(pt)//2], -10, color='black')
axs[0,0].text(pt[len(pt)//2], -9.95, 'ay = -10 m/s²', fontweight='bold')

# 2. VELOCIDAD vs TIEMPO
axs[0,1].plot(pt, pvy, 'b', linewidth=2)
axs[0,1].set_title('Velocidad vs Tiempo')
axs[0,1].set_ylabel('v (m/s)')
axs[0,1].set_xlabel('t (s)')
axs[0,1].grid(True)
# Punto v=0
axs[0,1].scatter(t_h_max, 0, color='black', zorder=5)
axs[0,1].text(t_h_max, 1, f'v=0\nt={t_h_max:.1f}s', ha='center', fontweight='bold')

# 3. ALTURA vs TIEMPO
axs[1,0].plot(pt, py, 'g', linewidth=2)
axs[1,0].set_title('Altura vs Tiempo')
axs[1,0].set_ylabel('y (m)')
axs[1,0].set_xlabel('t (s)')
axs[1,0].grid(True)
# Puntos clave y-t
axs[1,0].scatter([t_h_max, t_y2, t_suelo], [h_max, 2, 0], color='black', zorder=5)
axs[1,0].text(t_h_max, 6.2, f'Máx: {h_max:.2f}m\nt={t_h_max:.2f}s', ha='center', fontweight='bold')

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 13

```

    axs[1,0].text(t_y2, 2.3, f'y=2\nt={t_y2:.2f}s', ha='center', fontweight='bold')
    axs[1,0].text(t_suelo, 0.3, f'Suelo\nt={t_suelo:.2f}s', ha='center', fontweight='bold')

# 4. VELOCIDAD vs ALTURA
axs[1,1].plot(py, pvy, 'm', linewidth=2)
axs[1,1].set_title('Velocidad vs Altura')
axs[1,1].set_xlabel('y (m)')
axs[1,1].set_ylabel('v (m/s)')
axs[1,1].grid(True)
# Punto de retorno
axs[1,1].scatter(h_max, 0, color='black', zorder=5)
axs[1,1].text(6.1, 0, f'y={h_max:.2f}m\nv=0', ha='center', fontweight='bold')

# Respondiendo a las preguntas:
print("\nResultados:")
# (a) La altura máxima.
print(f"(a) Altura máxima: {h_max:.2f} m")
# (b) Cuando la velocidad sea cero.
print(f"(b) Velocidad cero en t = {t_h_max:.2f}s")
# (c) El tiempo donde llega a la altura máxima.
print(f"(c) Tiempo para altura máxima: {t_h_max:.2f} s")
# (d) El tiempo donde pasa por y = 2.
print(f"(d) Tiempo para y=2: {t_y2:.4f} s")
# (e) El tiempo en que llega al suelo.
print(f"(e) Tiempo para llegar al suelo: {t_suelo:.4f} s")

plt.tight_layout()
plt.savefig('TAREA-01/img/pr_a/exe211.png')
plt.show()

```

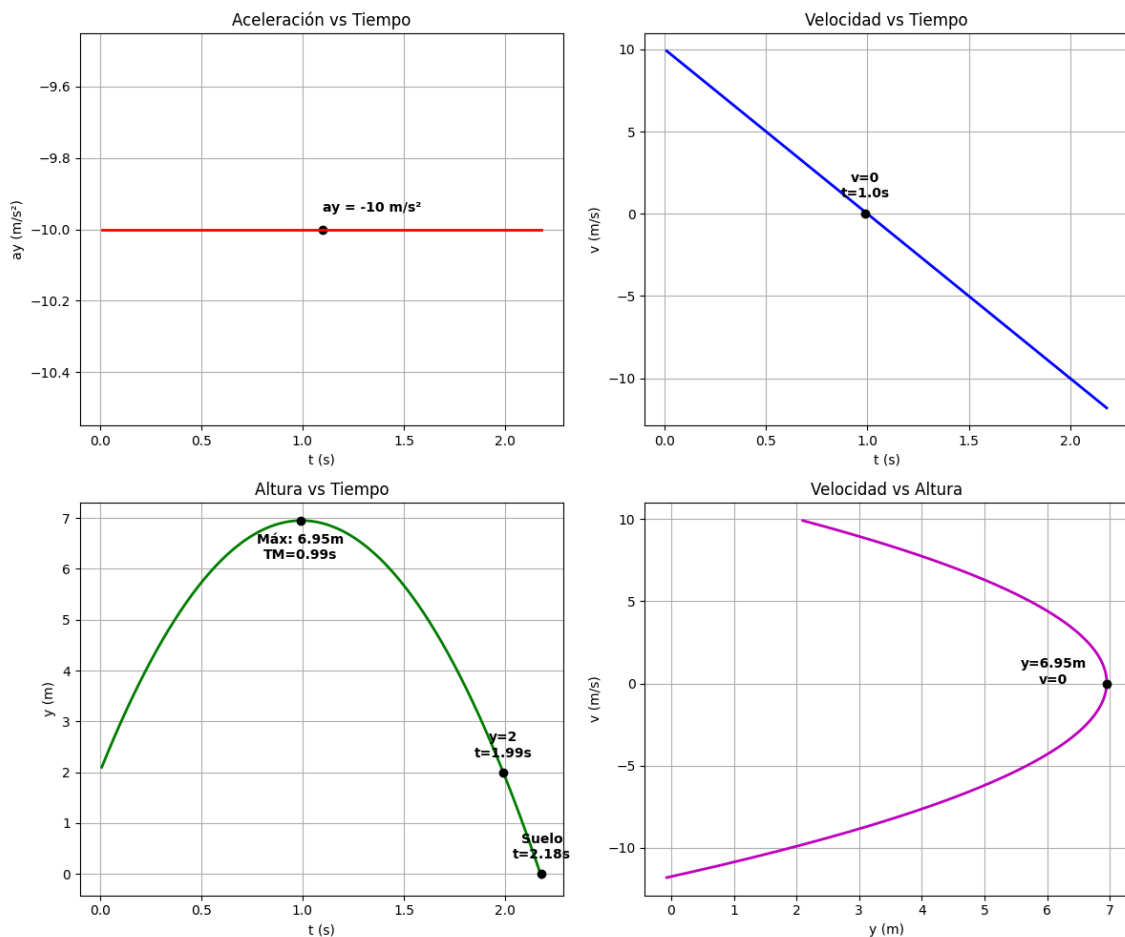


Figura 6: Diagramas $a-t$, $v-t$, $y-t$, $v-y$.

Descripción: En $a-t$ se observa una línea horizontal en $a_y = -10$ porque la gravedad es constante. En $v-t$ se observa una línea recta con pendiente negativa porque la velocidad disminuye debido a la gravedad. En $y-t$ se observa una parábola porque la pelota sube y baja. En $v-y$ se observa una parábola en forma de "C" invertida porque la velocidad cambia de dirección al alcanzar la altura máxima.

Resultado: Los resultados del ejercicio se observan en la siguiente tabla, algunos valores se redondearon a dos decimales para mayor claridad.

Parámetro	Valor	Unidad
Altura máxima	6.95	m
Velocidad=0	1.00	s
Tiempo altura máx	0.99	s
Tiempo pasa $y=2$	1.99	s
Tiempo suelo	2.18	s

Tabla 1: Resultados del ejercicio 2.1.1.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 15

2.2 En dos dimensiones. Movimiento parabólico

2.2.1 Problema

Una pelota se lanza desde el suelo con una velocidad de $v = 2 \text{ m/s}$ con un ángulo de 45° con la horizontal. Determine la altura máxima, la velocidad en esa altura, el alcance y el tiempo en que llega al suelo.

Implementación: Se calculó las componentes de velocidad inicial $v_x = v_0 \cos(\theta)$, $v_y = v_0 \sin(\theta)$. Se usó un ciclo while con v_y decreciente debido a la gravedad.

Código fuente: exe222.py

```
import numpy as np
import matplotlib.pyplot as plt

# 1. Constantes y condiciones iniciales
h = 0.001
t = 0
v0 = 2
angulo_deg = 45
angulo_rad = np.radians(angulo_deg)

# Componentes de la velocidad inicial
vx = v0 * np.cos(angulo_rad)
vy = v0 * np.sin(angulo_rad)
ay = -10

# Variables de estado inicial para el ciclo
x, y = 0, 0

# Listas para guardar datos
pt, px, py, pvy = [], [], [], []

# 2. Trayectoria
while y >= 0:
    pt.append(t)
    px.append(x)
    py.append(y)
    pvy.append(vy)

    # Euler: Actualización directa
    vy = vy + ay * h
    x = x + vx * h
    y = y + vy * h
    t += h

px, py, pt, pvy = np.array(px), np.array(py), np.array(pt), np.array(pvy)

# 3. Análisis de resultados
idx_max = np.argmax(py)
h_max = py[idx_max]
t_h_max = pt[idx_max]
v_max = vx

t_suelo = pt[-1]
x_alcance = px[-1]

# Etiquetas para los puntos clave
txt_hmax = f'H. máx: {h_max:.3f}m\nt={t_h_max:.2f}s'
txt_v_hmax = f'v=0\nt={t_h_max:.2f}s'
txt_res_b = f'v_x = {vx:.2f} m/s'
txt_alcance = f'Alcance: {x_alcance:.2f}m'
txt_suelo = f'T. suelo\nt={t_suelo:.2f}s'

# 4. Visualización
fig, axs = plt.subplots(2, 2, figsize=(12, 10))
plt.gcf().canvas.manager.set_window_title('Laboratorio: Movimiento Parabólico')
fig.suptitle(f'Análisis Cinemático ($v_0={v0}m/s$, $\theta={angulo_deg}^\circ$)', fontsize=16)

# 1. ACELERACIÓN vs TIEMPO
axs[0,0].plot(pt, [ay]*len(pt), 'r', linewidth=2)
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 16

```

axs[0,0].set_title('Aceleración vs Tiempo (Eje Y)')
axs[0,0].set_ylabel('ay (m/s²)')
axs[0,0].set_xlabel('t (s)')
axs[0,0].set_ylim(ay-2, ay+2)
axs[0,0].grid(True)
# Punto clave: ay constante
axs[0,0].scatter(pt[len(pt)//2], ay, color='black', zorder=5)
axs[0,0].text(pt[len(pt)//2], ay+0.2, f'ay = {ay} m/s²', ha='center', fontweight='bold')

# 2. VELOCIDAD VERTICAL vs TIEMPO
axs[0,1].plot(pt, pvy, 'b', linewidth=2)
axs[0,1].axhline(0, color='black', lw=1)
axs[0,1].set_title('Velocidad Vertical vs Tiempo')
axs[0,1].set_ylabel('vy (m/s)')
axs[0,1].set_xlabel('t (s)')
axs[0,1].grid(True)
# Punto v=0 (Altura máxima)
axs[0,1].plot(pt, pvy, 'b', linewidth=2)
axs[0,1].scatter(t_h_max, 0, color='black', zorder=5)
axs[0,1].text(t_h_max, 0.2, txt_v_hmax, ha='center', fontweight='bold')
# Velocidad horizontal constante
axs[0,1].plot(pt, [vx]*len(pt), 'orange', linestyle='--', linewidth=2, label='$v_x$ (Horizontal)')
axs[0,1].text(0.25, vx - 0.2, txt_res_b, ha='center', fontweight='bold', color='darkorange')

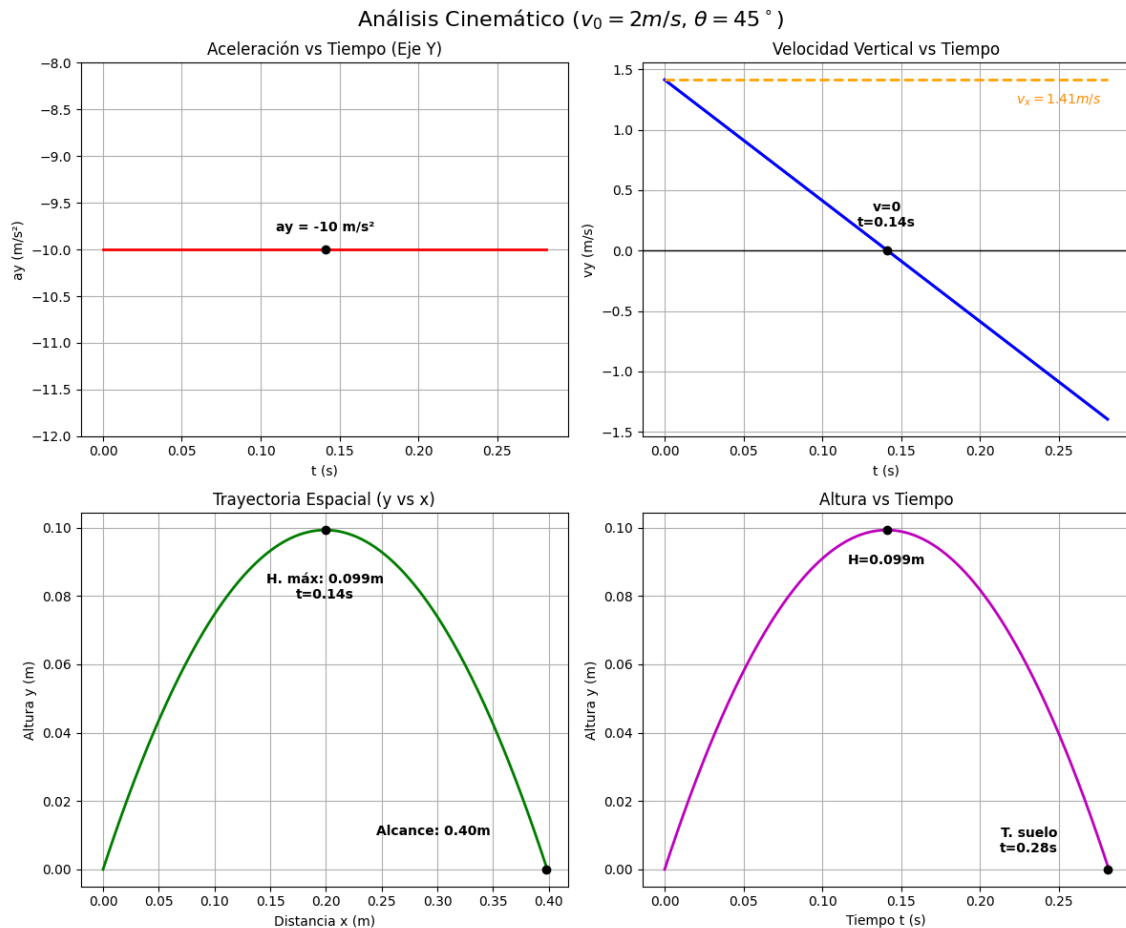
# 3. TRAYECTORIA ESPACIAL (y vs x)
axs[1,0].plot(px, py, 'g', linewidth=2)
axs[1,0].set_title('Trayectoria Espacial (y vs x)')
axs[1,0].set_xlabel('Distancia x (m)')
axs[1,0].set_ylabel('Altura y (m)')
axs[1,0].grid(True)
# Puntos clave: Altura máxima y alcance
axs[1,0].scatter([px[idx_max], x_alcance], [h_max, 0], color='black', zorder=5)
axs[1,0].text(px[idx_max], h_max-0.02, txt_hmax, ha='center', fontweight='bold')
axs[1,0].text(x_alcance-0.05, 0.01, txt_alcance, ha='right', fontweight='bold')

# 4. ALTURA vs TIEMPO
axs[1,1].plot(pt, py, 'm', linewidth=2)
axs[1,1].set_title('Altura vs Tiempo')
axs[1,1].set_xlabel('Tiempo t (s)')
axs[1,1].set_ylabel('Altura y (m)')
axs[1,1].grid(True)
# Puntos clave: t_h_max y t_suelo
axs[1,1].scatter([t_h_max, t_suelo], [h_max, 0], color='black', zorder=5)
axs[1,1].text(t_h_max, h_max-0.01, f'H={h_max:.3f}m', ha='center', fontweight='bold')
axs[1,1].text(t_suelo-0.05, 0.005, txt_suelo, ha='center', fontweight='bold')

# Responiendo a las preguntas:
print("Resultados:")
# (a) La altura máxima.
print(f"(a) Altura máxima: {h_max:.3f} m")
# (b) La velocidad en esa altura.
print(f"(b) Velocidad en altura máxima: {v_max:.2f} m/s (constante en x)")
# (c) El alcance.
print(f"(c) Alcance horizontal: {x_alcance:.2f} m")
# (d) El tiempo en que llega al suelo.
print(f"(d) Tiempo para llegar al suelo: {t_suelo:.2f} s")

plt.tight_layout()
plt.savefig('TAREA-01/img/pr_a/exe222.png')
plt.show()

```

Descripción: En a - t se observa $a_y = -10$ constante. En v_y - t se observa una línea con pendiente negativa. En y - x se observa una parábola porque es un movimiento parabólico. En y - t se observa una parábola.

Resultados: Los resultados se redondearon a dos decimales para mayor claridad.

Parámetro	Valor	Unidad
Altura máxima	0.10	m
Velocidad en H máx	1.41	m/s
Alcance	0.40	m
Tiempo suelo	0.28	s

2.3 En tres dimensiones. Movimiento parabólico

2.3.1 Problema

Una pelota se lanza desde el suelo desde una altura de $z = 1$ con $\vec{v} = (3\hat{i} + 4\hat{j} + 5\hat{k})$ m/s. Dibuje: la trayectoria y demuestre que es un parábola, la velocidad en esa altura, el alcance y el tiempo en que llega al suelo.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 18

Implementación: Se usaron las componentes vectoriales de velocidad (v_x , v_y , v_z). Se aplicó gravedad solo en eje Z ($a = -10$). Se representó la trayectoria 3D y se calculó el desplazamiento horizontal.

Código fuente: exe223.py

```
import numpy as np
import matplotlib.pyplot as plt

# 1. Constantes y condiciones iniciales
h = 0.001
t = 0

# Velocidad inicial vectorial v = (3i + 4j + 5k)
vx, vy, vz = 3, 4, 5
az = -10

# Variables de estado inicial
x, y, z = 0, 0, 1

# Listas para guardar datos
pt, px, py, pz, pvz = [], [], [], [], []

# 2. Trayectoria (Simulación hasta tocar suelo z >= 0)
while z >= 0:
    pt.append(t)
    px.append(x)
    py.append(y)
    pz.append(z)
    pvz.append(vz)

    # Euler: Actualización
    # vx y vy no cambian porque no hay aceleración en X ni Y
    vz = vz + az * h
    x = x + vx * h
    y = y + vy * h
    z = z + vz * h
    t += h

# Conversión a arrays
px, py, pz, pt, pvz = np.array(px), np.array(py), np.array(pz), np.array(pt), np.array(pvz)

# 3. Análisis de resultados
idx_max = np.argmax(pz)
h_max = pz[idx_max]
t_h_max = pt[idx_max]

v_en_hmax = np.sqrt(vx**2 + vy**2)

t_suelo = pt[-1]
alcance_horizontal = np.sqrt(px[-1]**2 + py[-1]**2)

# Etiquetas
txt_hmax = f'H. máx: {h_max:.2f}m\nt={t_h_max:.2f}s'
txt_suelo = f'Suelo\nt={t_suelo:.2f}s'
txt_vel_b = f'V en H.máx = {v_en_hmax:.2f} m/s'

# 4. Visualización
fig = plt.figure(figsize=(14, 10))
plt.gcf().canvas.manager.set_window_title('Laboratorio: Proyectil 3D')

# --- SUBPLOT 1: TRAYECTORIA 3D (Para demostrar la parábola) ---
ax1 = fig.add_subplot(2, 2, 1, projection='3d')
ax1.plot(px, py, pz, 'g', linewidth=3)
ax1.set_title('Trayectoria 3D (Demostración de Parábola)')
ax1.set_xlabel('Eje X (m)')
ax1.set_ylabel('Eje Y (m)')
ax1.set_zlabel('Altura Z (m)')
ax1.scatter(px[idx_max], py[idx_max], h_max, color='red', s=50) # Punto más alto

# --- SUBPLOT 2: VELOCIDAD VERTICAL (vz) vs TIEMPO ---
ax2 = fig.add_subplot(2, 2, 2)
ax2.plot(pt, pvz, 'b', linewidth=2)
ax2.axhline(0, color='black', lw=1)
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 19

```

ax2.set_title('Velocidad Vertical (vz) vs Tiempo')
ax2.set_ylabel('vz (m/s)')
ax2.grid(True)
ax2.scatter(t_h_max, 0, color='black')
ax2.text(t_h_max, 0.5, txt_vel_b, ha='center', fontweight='bold', color='blue')

# --- SUBPLOT 3: ALTURA (z) vs TIEMPO ---
ax3 = fig.add_subplot(2, 2, 3)
ax3.plot(pt, pz, 'm', linewidth=2)
ax3.set_title('Altura (z) vs Tiempo')
ax3.set_xlabel('t (s)')
ax3.set_ylabel('z (m)')
ax3.grid(True)
ax3.scatter([t_h_max, t_suelo], [h_max, 0], color='black')
ax3.text(t_h_max, h_max - 0.2, txt_hmax, ha='center', fontweight='bold')

# --- SUBPLOT 4: DISTANCIA HORIZONTAL vs TIEMPO ---
distancia_horizontal = np.sqrt(px**2 + py**2)
ax4 = fig.add_subplot(2, 2, 4)
ax4.plot(pt, distancia_horizontal, 'orange', linewidth=2)
ax4.set_title('Distancia Horizontal vs Tiempo')
ax4.set_xlabel('Tiempo (s)')
ax4.set_ylabel('Distancia (m)')
ax4.scatter(t_suelo, alcance_horizontal, color='black')
ax4.text(t_suelo-0.2, alcance_horizontal - 0.5, txt_suelo, ha='center', fontweight='bold', color='orange')
ax4.grid(True)

plt.tight_layout()

# Respuestas:
print(f"(a) La trayectoria es parabólica en el plano de movimiento.")
print(f"(b) Velocidad en z=1 m: vz = {pvz[0]:.2f} m/s")
print(f"(c) El alcance horizontal es de {alcance_horizontal:.2f} m")
print(f"(d) Tiempo hasta el suelo: {t_suelo:.2f} s")

plt.savefig('TAREA-01/img/pr_a/exe223.png')
plt.show()

```

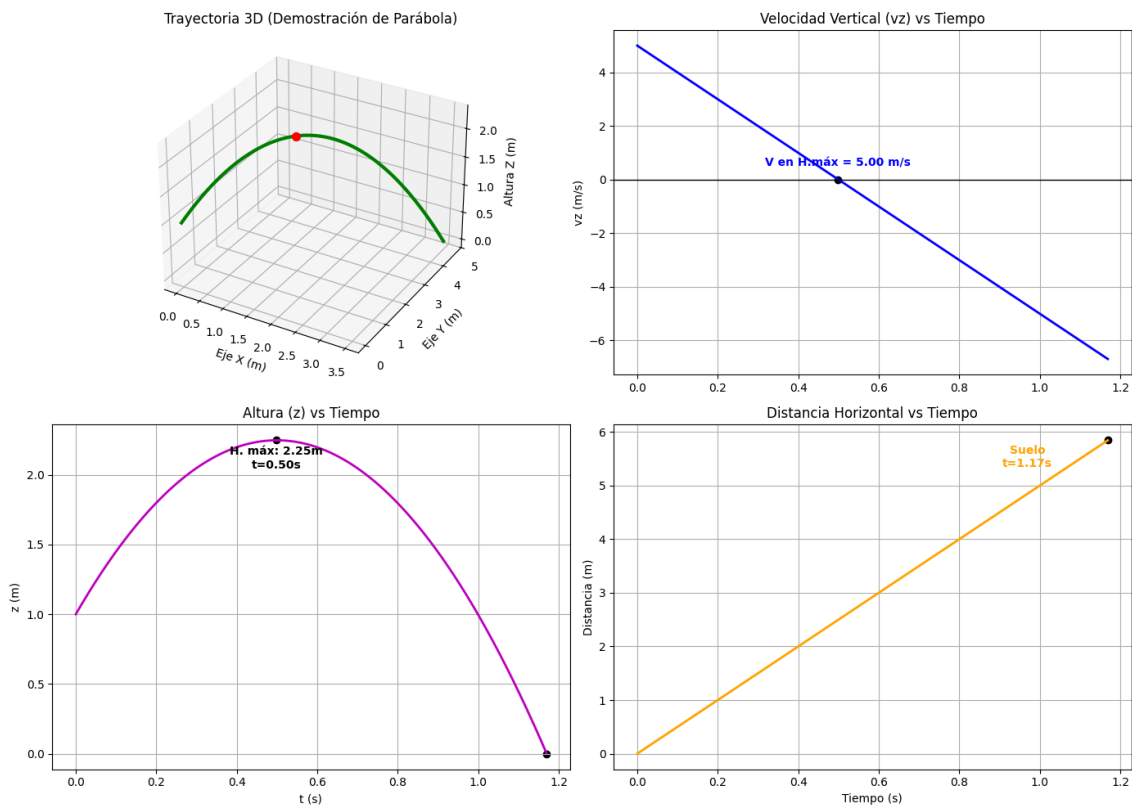


Figura 8: Trayectoria en tres dimensiones.

Descripción: En $a-t$ se observa una línea horizontal en $az = -10$ porque la gravedad es constante. En $vz-t$ se observa una línea con pendiente negativa. En $y-x$ se observa una línea recta porque no hay movimiento en el eje Y. En $z-x$ se observa una parábola porque es un movimiento parabólico en el plano XZ.

La trayectoria es una parábola porque el movimiento en el eje Z está influenciado por la gravedad, mientras que el movimiento en el eje X es uniforme. Al combinar ambos movimientos, la trayectoria resultante describe una curva parabólica en el espacio tridimensional.

Resultados: Como vemos en el gráfico, la trayectoria

Parámetro	Valor	Unidad
Velocidad en $z=1$ m	5.00	m/s
Alcance horizontal	5.84	m
Tiempo suelo	1.17	s

CONCLUSIONES Y RECOMENDACIONES

CONCLUSIONES

1. Se logró implementar exitosamente los ejercicios de cinemática, desde el movimiento con aceleración cero hasta el movimiento parabólico en tres dimensiones, verificando que los resultados teóricos coinciden con las simulaciones numéricas realizadas.
2. El método de Euler para resolver las ecuaciones de movimiento con paso de tiempo h mostró ser efectivo para los casos estudiados, permitiendo visualizar las trayectorias y analizar el comportamiento de cada variable cinemática (posición, velocidad y aceleración) en función del tiempo.
3. El análisis gráficas permite comprender de manera más intuitiva el comportamiento cinemático de las partículas, donde se pudo observar que la velocidad constante. Produce movimientos rectilíneos, mientras que con aceleración constante produce movimientos curvilíneos como la parábola.

RECOMENDACIONES

1. Se recomienda no combinar líneas de código de lógica con las de hacer los diagramas, ya que esto dificulta la lectura y depuración del programa. Es mejor separar claramente las secciones de cálculo y graficado para cada ejercicio.